

# Exercice de groupe – Backend Node.js

## Mini-gestionnaire de bibliothèque

Prisma + Neon + TypeScript

---

Cours 420-941-MA • Services Web • Séance 3 – Atelier pratique

Collège de Maisonneuve • Groupe 25604

---

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <b>Durée :</b>      | 3 heures (en classe + finalisation)                   |
| <b>Équipes :</b>    | 3 personnes maximum                                   |
| <b>Outils :</b>     | Node.js LTS, VS Code, navigateur, Postman (optionnel) |
| <b>Stack :</b>      | Node.js + TypeScript + Prisma + PostgreSQL (Neon)     |
| <b>Évaluation :</b> | Formative (pas notée), mais à présenter en classe     |

## 1. Contexte

La bibliothèque du Collège de Maisonneuve veut remplacer son vieux tableur Excel par un vrai système informatique. En équipe, vous allez bâtir un **mini-backend Node.js** qui se connecte à une base PostgreSQL hébergée sur Neon (via Prisma) et qui sait gérer un catalogue de livres et leurs emprunts.

L'objectif n'est **pas** de bâtir une vraie API web (pas d'Express ni de Next.js ici) : on se concentre sur la **couche d'accès aux données**, brique fondamentale de tout backend.

## 2. Objectifs pédagogiques

### Objectif

À la fin de cet exercice, vous saurez :

- Initialiser un projet Node.js + TypeScript propre.
- Créer une base PostgreSQL sur Neon (sans rien installer localement).
- Décrire un schéma de données avec Prisma (deux modèles liés).
- Configurer le client Prisma avec l'adapter Neon dans `utils/prisma.ts`.
- Implémenter les 4 opérations **CRUD** : Create, Read, Update, Delete.
- Faire des requêtes filtrées et utiliser une relation entre deux tables.

## 3. Prérequis

- Node.js LTS installé (`node -v` et `npm -v` doivent répondre).
- VS Code (ou un autre éditeur).
- Un compte Neon (gratuit) : <https://neon.tech>.
- Avoir suivi (ou révisé) la séance 3.

## 4. Le scénario en bref

Trois entités à manipuler :

- **Livre** : un livre du catalogue (titre, auteur, année, disponibilité).
- **Emprunt** : un livre emprunté par un étudiant (qui, quoi, quand).

— **Relation** : un livre peut avoir plusieurs emprunts (1-N).

#### Astuce

Travaillez en équipe : un membre clone, un autre rédige le schéma, un troisième écrit les fonctions CRUD. **Tout le monde doit comprendre tout le code** : on se relit avant de remettre.

## Diagramme des entités

Voici la représentation visuelle des deux modèles et de leur relation. Vous traduirez ce diagramme en `schema.prisma` à l'étape 1.

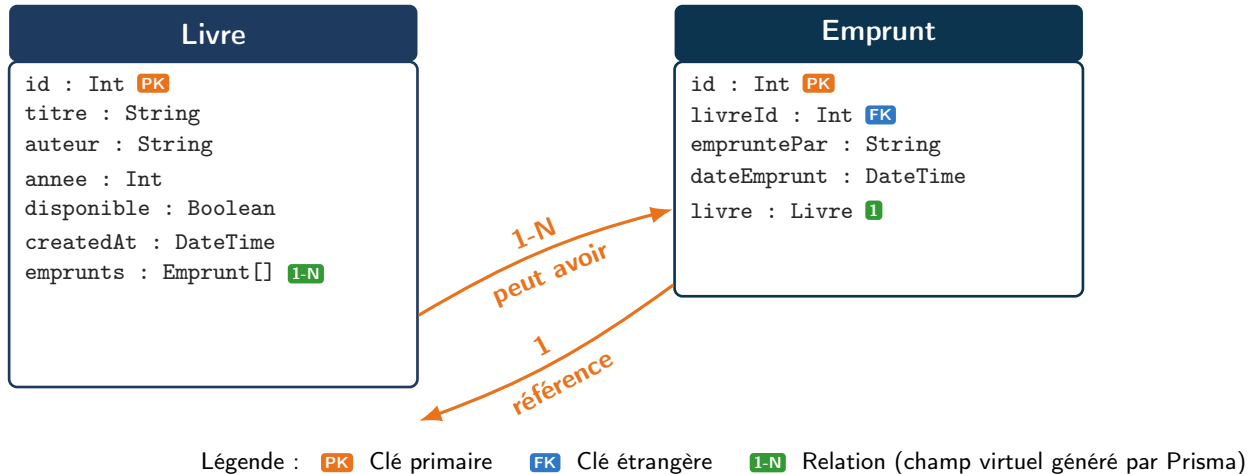


FIGURE 1 – Diagramme des entités Livre et Emprunt (relation 1-N).

**Lecture du diagramme** : chaque Livre peut avoir plusieurs Emprunt (relation 1-N). Chaque Emprunt pointe vers exactement un Livre via la clé étrangère `livreId`. Les champs `emprunts` et `livre` sont des *relations* : ils ne sont pas stockés en BD, ils sont reconstruits par Prisma pour faciliter les requêtes.

## 5. Découpage suggéré (3 heures)

| primary !12 Étape | Contenu                                     | Durée          |
|-------------------|---------------------------------------------|----------------|
| 0                 | Mise en place du projet (Node + TS + Neon)  | 25 min         |
| 1                 | Schéma Prisma (Livre + Emprunt) + migration | 20 min         |
| 2                 | Le fichier <code>utils/prisma.ts</code>     | 15 min         |
| 3                 | CREATE : ajouter des livres                 | 30 min         |
| 4                 | READ : lire et filtrer les livres           | 30 min         |
| 5                 | UPDATE : modifier un livre                  | 20 min         |
| 6                 | DELETE : supprimer un livre                 | 15 min         |
| B                 | BONUS : emprunts (relation 1-N)             | 25 min         |
|                   | <b>Total</b>                                | <b>180 min</b> |

## 6. Étape 0 – Mise en place du projet (25 min)

### Objectif

Avoir un projet Node.js + TypeScript prêt à recevoir Prisma, et une base PostgreSQL vide hébergée sur Neon.

### Rappel des étapes

Suivez les étapes **dans cet ordre** :

1. Créer un dossier de projet et initialiser npm.
2. Installer les packages (dev + production).
3. Initialiser TypeScript (`tsconfig.json`).
4. Créer un compte sur <https://neon.tech> (si pas déjà fait).
5. Créer un nouveau projet Neon → copier la `DATABASE_URL`.
6. Coller cette URL dans un fichier `.env` à la racine.
7. Ajouter `.env` et `node_modules/` dans `.gitignore`.

Commandes à taper :

Listing 1 – Initialisation du projet

```
mkdir biblio-api && cd biblio-api
npm init -y

# Outils de developpement
npm install -D prisma typescript tsx @types/node

# Production
npm install @prisma/client @prisma/adapters-neon dotenv

# Initialiser TypeScript et Prisma
npx tsc --init
npx prisma init
```

Modifications à apporter manuellement dans `package.json` :

Listing 2 – `package.json` (extrait)

```
1 {
2   "type": "module",
3   "scripts": {
4     "dev": "tsx src/index.ts",
5     "db:migrate": "prisma migrate dev",
6     "db:studio": "prisma studio",
7     "db:generate": "prisma generate"
8   }
9 }
```

Fichier `.env` (à la racine, à NE PAS pousser sur Git) :

Listing 3 – `.env`

```
DATABASE_URL="postgresql://USER:PASSWORD@ep-xxxx.us-east-2.aws.neon.tech/neondb?sslmode=require"
```

Fichier `.gitignore` :

Listing 4 – `.gitignore`

```
node_modules/
```

```
.env
generated/
```

### Attention

Ne partagez jamais votre `.env` sur GitHub, Discord, etc. Si vous le faites par erreur, retournez sur Neon et cliquez sur *Reset password* pour générer une nouvelle URL.

## 7. Étape 1 – Schéma Prisma (20 min)

### Objectif

Décrire vos deux tables `Livre` et `Emprunt` dans `prisma/schema.prisma`, puis créer les tables dans Neon avec une migration.

### Rappel des étapes

1. Ouvrir `prisma/schema.prisma`.
2. Configurer le bloc `generator` avec un `output` custom + `driverAdapters`.
3. Vérifier que `datasource db` pointe bien sur `env("DATABASE_URL")`.
4. Définir les modèles `Livre` et `Emprunt`.
5. Lancer la migration : `npx prisma migrate dev -name init`.
6. Vérifier dans Neon (onglet Tables) que les deux tables existent.

Contenu attendu de `schema.prisma` :

Listing 5 – `prisma/schema.prisma`

```
1  generator client {
2    provider      = "prisma-client-js"
3    output        = "../generated/prisma/client"
4  }
5
6  datasource db {
7    provider = "postgresql"
8    url      = env("DATABASE_URL")
9  }
10
11 model Livre {
12   id          Int           @id @default(autoincrement())
13   titre       String
14   auteur      String
15   annee       Int
16   disponible Boolean @default(true)
17   createdAt   DateTime @default(now())
18   emprunts    Emprunt[]
19 }
20
21 model Emprunt {
22   id          Int           @id @default(autoincrement())
23   livreId     Int
24   empruntePar String
25   dateEmprunt DateTime @default(now())
26   livre       Livre        @relation(fields: [livreId], references: [id])
27 }
```

### Astuce

Si vous modifiez le schéma plus tard, relancez `npx prisma migrate dev -name <nom>` et `npx prisma generate` pour mettre à jour les tables ET le client.

## 8. Étape 2 – Le fichier `utils/prisma.ts` (15 min)

### Objectif

Créer **une seule instance partagée** du client Prisma, configurée avec l'adapter Neon, exportée pour être utilisée partout dans le projet.

### Rappel des étapes

1. Créer un dossier `utils/` à la racine.
2. Y créer un fichier `prisma.ts`.
3. Importer `PrismaNeon`, `PrismaClient`, `dotenv`.
4. Charger `.env` avec `dotenv.config()`.
5. Créer l'adapter avec la `DATABASE_URL`.
6. Créer le client Prisma avec l'adapter + le logging.
7. Exporter le client par défaut.

Listing 6 – `utils/prisma.ts`

```
1 // utils/prisma.ts
2 import { PrismaNeon } from "@prisma/adapter-neon";
3 import { PrismaClient } from "../generated/prisma/client";
4 import dotenv from "dotenv";
5 dotenv.config();
6
7 const adapter = new PrismaNeon({
8   connectionString: process.env.DATABASE_URL,
9 });
10
11 const prisma = new PrismaClient({
12   adapter,
13   log: ["query", "info", "warn", "error"],
14 });
15
16 export default prisma;
```

### Astuce

À partir de maintenant, dans n'importe quel fichier de votre projet : `import prisma from "../utils/prisma"`; et c'est tout.

## 9. Étape 3 – CREATE : ajouter des livres (30 min)

### Objectif

Créer un fichier `src/index.ts` et y ajouter au moins 5 livres dans la base (dont au moins 2 indisponibles).

### Rappel des étapes

1. Créer un dossier `src/` et un fichier `src/index.ts`.
2. Importer `prisma` depuis `../utils/prisma`.
3. Écrire une fonction `async function seed()`.
4. Utiliser `prisma.livre.create({ data: { ... } })` ou `createMany`.
5. Lancer le script avec `npm run dev`.
6. Vérifier le résultat avec `npm run db:studio`.

Squelette de départ :

Listing 7 – `src/index.ts` (à compléter)

```
1 // src/index.ts
2 import prisma from "../utils/prisma";
3
4 async function seed() {
5   // TODO : ajouter au moins 5 livres
6   const livre1 = await prisma.livre.create({
7     data: {
8       titre: "Le Petit Prince",
9       auteur: "Antoine de Saint-Exupéry",
10      annee: 1943,
11      disponible: true,
12    },
13  });
14  console.log("Créé :", livre1);
15
16  // À COMPLÉTER : 4 autres livres minimum
17 }
18
19 async function main() {
20   await seed();
21   await prisma.$disconnect();
22 }
23
24 main().catch((e) => {
25   console.error(e);
26   process.exit(1);
27 });
```

### Astuce

`prisma.livre.createMany({ data: [...] })` permet de créer plusieurs livres en une seule requête. Plus rapide pour du seed.

**Vérification** : ouvrez Prisma Studio (`npm run db:studio`) et confirmez que la table `Livre` contient bien vos 5 livres.

## 10. Étape 4 – READ : lire et filtrer les livres (30 min)

### Objectif

Implémenter 4 fonctions de lecture, chacune avec un usage différent de Prisma.

## Rappel des étapes

1. `findMany()` : tout lire.
2. `findMany({ where: ... })` : filtrer.
3. `findUnique({ where: { id } })` : un seul.
4. Utiliser `contains` pour une recherche partielle.
5. Afficher les résultats avec `console.log`.

Fonctions à écrire dans `src/index.ts` (ou un fichier séparé `src/lectures.ts`) :

Listing 8 – Les 4 fonctions de lecture

```
1 // 1) Tous les livres
2 async function getTousLesLivres() {
3   return prisma.livre.findMany();
4 }
5
6 // 2) Seulement les livres disponibles
7 async function getLivresDisponibles() {
8   return prisma.livre.findMany({
9     where: { disponible: true },
10  });
11 }
12
13 // 3) Un livre par son id
14 async function getLivreParId(id: number) {
15   return prisma.livre.findUnique({
16     where: { id },
17  });
18 }
19
20 // 4) Recherche partielle par auteur
21 async function chercherParAuteur(motCle: string) {
22   return prisma.livre.findMany({
23     where: {
24       auteur: { contains: motCle, mode: "insensitive" },
25     },
26  });
27 }
```

Test à exécuter dans la fonction `main()` :

Listing 9 – `main()` - bloc de test

```
1 async function main() {
2   console.log("\n--- Tous les livres ---");
3   console.log(await getTousLesLivres());
4
5   console.log("\n--- Livres disponibles ---");
6   console.log(await getLivresDisponibles());
7
8   console.log("\n--- Livre #1 ---");
9   console.log(await getLivreParId(1));
10
11  console.log("\n--- Recherche : 'saint' ---");
12  console.log(await chercherParAuteur("saint"));
13
14  await prisma.$disconnect();
15 }
```

### Astuce

Tapez `prisma.livre.` dans VS Code : l'auto-complétion vous propose toutes les méthodes disponibles. Idem pour les champs (`titre`, `auteur`, etc.). C'est la magie Prisma + TypeScript.

## 11. Étape 5 – UPDATE : modifier un livre (20 min)

### Objectif

Écrire deux fonctions de mise à jour : changer la disponibilité, corriger une donnée erronée.

### Rappel des étapes

1. Utiliser `prisma.livre.update`.
2. Toujours fournir un `where` pour cibler la ligne.
3. Fournir un `data` avec les champs à modifier.
4. Si l'id n'existe pas → une erreur est levée (à attraper avec `try/catch`).

Listing 10 – Fonctions de mise à jour

```
1 // Marquer un livre comme indisponible (emprunté)
2 async function marquerIndisponible(id: number) {
3   return prisma.livre.update({
4     where: { id },
5     data: { disponible: false },
6   });
7 }
8
9 // Corriger l'année d'un livre
10 async function corrigerAnnee(id: number, nouvelleAnnee: number) {
11   return prisma.livre.update({
12     where: { id },
13     data: { annee: nouvelleAnnee },
14   });
15 }
```

Test :

Listing 11 – Test des updates

```
1 console.log(await marquerIndisponible(1));
2 console.log(await corrigerAnnee(2, 2024));
```

### Astuce

Pour modifier plusieurs lignes en même temps, utilisez `updateMany`. Exemple : rendre tous les livres disponibles d'un coup.

## 12. Étape 6 – DELETE : supprimer un livre (15 min)

### Objectif

Implémenter deux fonctions de suppression : suppression simple et suppression conditionnelle (plusieurs lignes).



### Rappel des étapes

1. `prisma.livre.delete({ where: { id } })` pour un.
2. `prisma.livre.deleteMany({ where: ... })` pour plusieurs.
3. **Attention** : si une ligne est référencée par une FK (un emprunt), Prisma refusera (sauf si `onDelete: Cascade` dans le schéma).

Listing 12 – Fonctions de suppression

```
1 // Supprimer un livre par id
2 async function supprimerLivre(id: number) {
3   return prisma.livre.delete({
4     where: { id },
5   });
6 }
7
8 // Supprimer tous les livres antérieurs à une année
9 async function supprimerAnciens(avantAnnee: number) {
10  return prisma.livre.deleteMany({
11    where: { annee: { lt: avantAnnee } },
12  });
13 }
```

### Attention

`deleteMany` ne demande aucune confirmation : une fois lancé, les lignes sont parties. Faites une sauvegarde de votre seed avant de l'essayer.

## 13. Bonus – Les emprunts et les relations (25 min)

### Objectif

Manipuler la relation 1-N entre Livre et Emprunt : créer un emprunt et lire les emprunts avec le titre du livre associé.

### Rappel des étapes

1. Créer un emprunt → `prisma.emprunt.create` avec `livreId`.
2. Mettre à jour le livre → `disponible: false`.
3. Lire les emprunts avec leur livre → `include: { livre: true }`.
4. Rendre un livre → supprimer l'emprunt + remettre `disponible: true`.

Listing 13 – Fonctions emprunt

```
1 // Emprunter un livre
2 async function emprunterLivre(livreId: number, parQui: string) {
3   // 1) Créer l'emprunt
4   const emprunt = await prisma.emprunt.create({
5     data: { livreId, empruntePar: parQui },
6   });
7   // 2) Marquer le livre comme indisponible
8   await prisma.livre.update({
9     where: { id: livreId },
10    data: { disponible: false },
11  });
12  return emprunt;
13 }
14
15 // Lister tous les emprunts AVEC les infos du livre
```

```

16  async function listerEmprunts() {
17    return prisma.emprunt.findMany({
18      include: { livre: true },
19    });
20  }
21
22  // Retourner un livre (rendre l'emprunt)
23  async function rendreLivre(empruntId: number) {
24    const emprunt = await prisma.emprunt.delete({
25      where: { id: empruntId },
26    });
27    await prisma.livre.update({
28      where: { id: emprunt.livreId },
29      data: { disponible: true },
30    });
31    return emprunt;
32  }

```

### Astuce

Vous pouvez aussi inclure les emprunts dans la requête sur les livres :  
`prisma.livre.findMany({ include: { emprunts: true } })`.

## 14. Livrables

### À livrer

À la fin de l'exercice, votre équipe doit fournir :

1. Le **code source complet** dans un dépôt GitHub privé partagé avec le prof.
2. Le fichier `schema.prisma` avec les deux modèles.
3. Le fichier `utils/prisma.ts` configuré pour Neon.
4. Le fichier `src/index.ts` démontrant les 4 opérations CRUD (au minimum).
5. Un `README.md` court expliquant : ce que fait le projet, comment l'installer, comment le lancer.
6. Une **capture d'écran de Prisma Studio** montrant vos données.
7. Les noms des membres de l'équipe dans le `README`.

**Ne PAS pousser** : `.env`, `node_modules/`, `generated/`.

## 15. Critères d'auto-évaluation

| primary!12 Critère                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Validé ? |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Le projet démarre sans erreur ( <code>npm run dev</code> )<br>La migration Prisma a créé les deux tables sur Neon<br><code>utils/prisma.ts</code> fonctionne et utilise l'adapter Neon<br>Au moins 5 livres sont créés via le script de seed<br>Les 4 fonctions de lecture renvoient les bons résultats<br><code>update</code> modifie correctement un livre<br><code>delete</code> supprime correctement un livre<br>Tout le code est en TypeScript ( <code>.ts</code> , pas <code>.js</code> )<br><code>.env</code> et <code>node_modules</code> sont dans <code>.gitignore</code><br>Chaque membre de l'équipe peut expliquer tout le code<br><b>Bonus</b> : emprunts avec relation <code>include</code> |          |

## 16. Dépannage rapide

| primary!12 Message d'erreur                     | Cause probable et solution                                                                                                         |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Can't reach database server                     | La <code>DATABASE_URL</code> est mauvaise OU le projet Neon est en pause. Vérifier sur <a href="https://neon.tech">neon.tech</a> . |
| Cannot find module '../generated/prisma/client' | Le client n'a pas été généré. Lancer <code>npx prisma generate</code> .                                                            |
| PrismaClientValidationError                     | Un champ envoyé dans <code>data</code> n'existe pas dans le schéma. Vérifier l'orthographe.                                        |
| ERR_MODULE_NOT_FOUND                            | Manque <code>"type": "module"</code> dans <code>package.json</code> .                                                              |
| Foreign key constraint failed                   | On essaye de supprimer un livre encore référencé par un emprunt. Supprimer d'abord l'emprunt.                                      |
| Erreur TypeScript sur les imports               | Vérifier que <code>tsx</code> est installé et qu'on lance avec <code>npx tsx</code> ou <code>npm run dev</code> .                  |

## Mot de la fin

« Construisez, cassez, corrigez, documentez et déployez.  
C'est en livrant qu'on devient développeur. »

**Bon atelier !**